# ICT2403 – Video-Based Road Damage Detection System

## 1. Introduction

- Road surface deterioration such as cracks, potholes, and surface wear is a major issue affecting transportation safety and infrastructure maintenance. Manual inspection of road conditions is time-consuming, expensive, and inefficient. Therefore, developing an automated video-based image processing system for road damage detection is important.

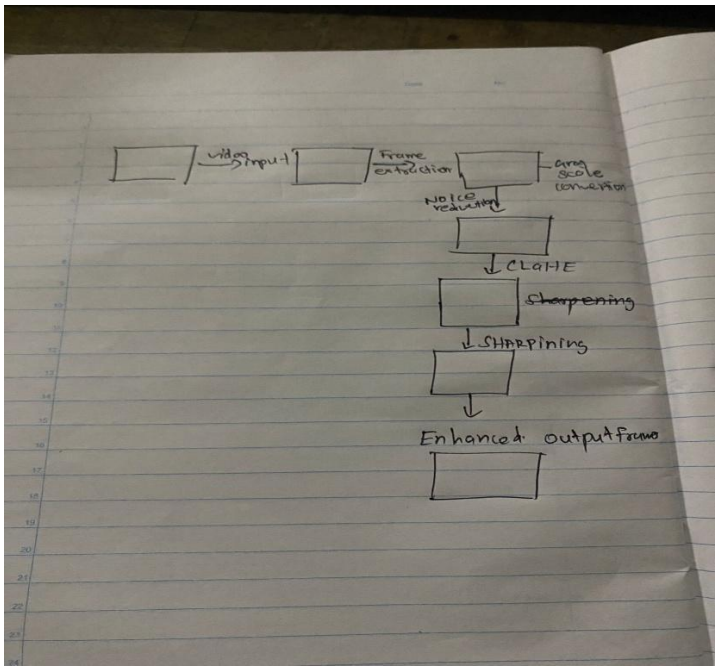
- However, real-world road videos often suffer from image degradation problems including sensor noise, compression artefacts, uneven illumination, shadows, and motion blur. These issues reduce the visibility of cracks and negatively affect segmentation and detection accuracy.

- This project aims to develop a complete video enhancement pipeline to improve road surface visibility before segmentation. The system includes noise analysis, illumination correction, motion blur handling, and overall pipeline integration. The effectiveness of the enhancement methods is evaluated using histogram analysis, entropy metrics, and visual comparison

# 2. Methods

## 2.1 Overall System Pipeline

The proposed system consists of the following stages:



## 2.2 Frame Extraction and Dataset Preparation (Member 4)

- The input road video was loaded using OpenCV.
- Total frame count was obtained.
- 30 unique random frames were selected.
- Frames were stored for further processing.

**Figure 1**

```
[1]
#Import Libraries
import cv2
import os
import numpy as np
import random
import matplotlib.pyplot as plt

[3]
#Get Total Frames
video_path = "vdo1.mp4"
cap = cv2.VideoCapture(video_path)

total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
print("Total Frames:", total_frames)

cap.release()

Total Frames: 633

[4]
#Randomly Select 30 Unique Frame Numbers
random_frames = random.sample(range(total_frames), 30)
random_frames.sort()

print(random_frames)

[14, 29, 37, 40, 48, 50, 63, 90, 143, 145, 181, 182, 230, 239, 283, 305, 306, 315, 322, 382, 385, 413, 458, 486, 555, 559, 581, 613, 622, 632
```

```
#Extract Those 30 Frames
output_folder = "random_30_frames"

if not os.path.exists(output_folder):
    os.makedirs(output_folder)

cap = cv2.VideoCapture(video_path)

current_frame = 0
saved_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    if current_frame in random_frames:
        resized = cv2.resize(frame, (640, 480))
        cv2.imwrite(f"{output_folder}/frame_{saved_count}.jpg", resized)
        saved_count += 1

    current_frame += 1

cap.release()

print("Random 30 Frames Saved:", saved_count)

Random 30 Frames Saved: 30
```

## 2.3 Noise Analysis and Removal (Member 1)

**Noise Types Considered:**

- Sensor noise
- Compression noise

Histogram inspection and visual analysis were performed.

## Noise Reduction Technique

Gaussian Filtering

Gaussian filter smooths high-frequency noise while preserving structure.

Gaussian equation:

$$G(x,y) = 1 / (2\pi\sigma^2) \exp(-(x^2 + y^2) / 2\sigma^2)$$

Median filtering was also tested for comparison.

Effect:

- Reduced random intensity fluctuations
- Improved smoothness before contrast enhancement

```
img = cv2.imread("frames/frame_0.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply filters on grayscale directly
gaussian = cv2.GaussianBlur(gray, (5,5), 0)
median = cv2.medianBlur(gray, 5)
```

### 2.4 Illumination Variation Analysis (Member 2)

This section focuses on correcting uneven lighting, shadows, and low exposure conditions.

**Method 1 – Contrast Stretching**

Global contrast enhancement was applied using:

New Pixel = (Pixel − Min) / (Max − Min) × 255

This expands the dynamic range of the image.

Limitation:

- Does not handle local shadow regions effectively.

**Method 2 – CLAHE (Contrast Limited Adaptive Histogram Equalization)**

CLAHE divides the image into small tiles and applies histogram equalization locally.

Parameters used:

- Clip Limit = 2.0
- Tile Grid Size = 8 × 8

Advantages:

- Enhances local contrast
- Improves visibility in shadow areas
- Prevents noise over-amplification

```
#Method 1 - Contrast Stretching
min_val = np.min(gray)
max_val = np.max(gray)

stretched = ((gray - min_val) / (max_val - min_val)) * 255
stretched = stretched.astype(np.uint8)

#Method 2 - CLAHE
clahe = cv2.createCLAHE(2.0, (8,8))
enhanced = clahe.apply(gray)
```

### 2.5 Motion Blur and Sharpening (Member 3)

Motion blur and camera instability were analyzed using frame sharpness evaluation.

**Sharpening Method**

Laplacian-based edge enhancement

Sharpened Image = Original − α × Laplacian

This improves crack edge visibility and structural clarity.

```
# Sharpening
laplacian = cv2.Laplacian(clahe_img, cv2.CV_64F)
sharpened = clahe_img - 0.5 * laplacian
sharpened = np.clip(sharpened, 0, 255)
sharpened = np.uint8(sharpened)
```

## 3. Results

### 3.1 Visual Comparison

For selected representative frames:

Original | Noise Reduced | contrast (CLAHE) | Sharpened

Observations:

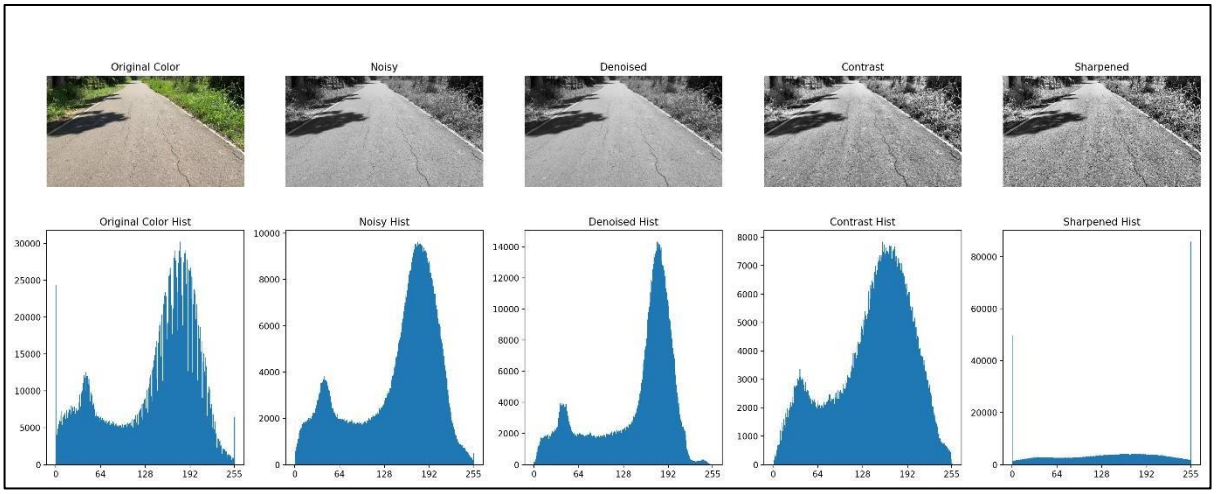
- Noise reduction improved smoothness.
- CLAHE significantly enhanced local contrast.

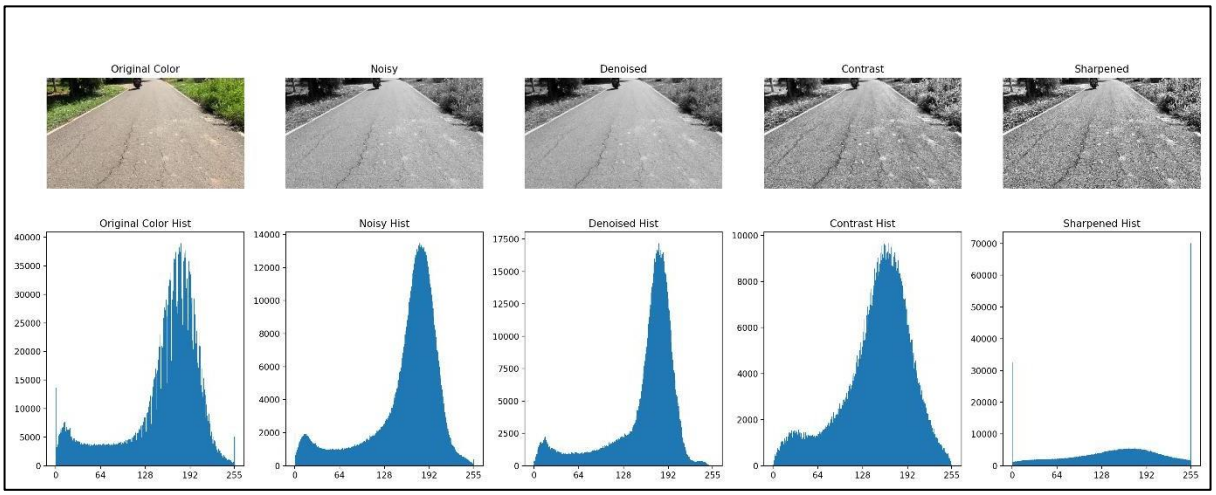
- Sharpening improved edge visibility.

## 3.2 Histogram Analysis

Histogram plots show:

- Original frames had narrow intensity distributions.
- Contrast stretching expanded global intensity range.
- CLAHE redistributed intensities more uniformly.
- Sharpening increased high-frequency components.



### 3.3 Entropy Evaluation

Entropy was calculated to measure information content:

$$\text{Entropy} = -\Sigma\, p(x) \log_2 p(x)$$

Results indicate:

- Entropy increased after enhancement.
- CLAHE produced higher entropy compared to contrast stretching.
- Increased entropy confirms improved contrast and detail representation.

```python
def calculate_entropy(image):
    histogram, _ = np.histogram(image.flatten(), bins=256, range=(0,256))
    histogram = histogram / histogram.sum()
    histogram = histogram[histogram > 0]
    return -np.sum(histogram * np.log2(histogram))

entropy_original = calculate_entropy(gray)
entropy_stretch = calculate_entropy(stretched)
entropy_clahe = calculate_entropy(enhanced)

print("Entropy Comparison")
print("Original:", entropy_original)
print("Contrast Stretch:", entropy_stretch)
print("CLAHE:", entropy_clahe)
```

```
Entropy Comparison
Original: 7.002118632235925
Contrast Stretch: 7.002118632235925
CLAHE: 7.777167588545013
```

The full enhancement pipeline significantly improved visibility of road surface features, which is essential for the next segmentation stage.

## 4. Discussion

- The experimental results demonstrate that the proposed enhancement pipeline effectively improves road surface visibility under varying illumination conditions.

- Gaussian filtering reduced noise while preserving structural information. CLAHE significantly improved local contrast compared to global methods. Sharpening enhanced edge clarity, which is important for crack detection.

- Although the pipeline improved image quality, fixed parameters may not adapt to extreme lighting variations. Future improvements may include adaptive parameter tuning and integration with segmentation algorithms for automated road damage detection.

## 5. References


→ OpenCV Documentation.

→ NumPy Documentation.

→ Matplotlib Documentation.


## 6. Individual contribution page

| Index Number | Name | Contribution |
|---|---|---|
| 6082 | P.P.K.Bhagya | Overall degradation summary and dataset preparation |
| 6083 | S. Pavatharan | Motion blur and sharpening |
| 6084 | MI.Fathima Nuha | Illumination analysis, CLAHE implementation, entropy evaluation. |
| 6085 | MF. Ahamed Afnan | Noise analysis |

## 7. GitHub Repository

https://github.com/MIFN08/Image_Enhancement.git